

EXPERIMENT - 27

1. Problem Statement

Write a C program that displays a menu and uses a switch statement to print the "Pyramid/Diamond pattern" selected by the user. The patterns have to be made with nested loops without using arrays, and each pattern may be written in a separate parameterless function with no return value.

- **Full pyramid**, input the number of rows n and print a full pyramid using asterisks.
- **Inverted pyramid**, input n and print an inverted pyramid using asterisks.
- **Diamond pattern**, input n for the upper half and print a diamond using asterisks. The middle row contains $(2n - 1)$ asterisks and the complete diamond contains $(2n - 1)$ rows.
- **Palindromic number pyramid**, input n . Each row prints the natural numbers in increasing order from 1 to the row number and then in decreasing order back to 1.
- **Repeated-number pyramid**, input n . Each row prints the row number $(2n - 1)$ times, where n denotes the current row number.

2. Algorithm

Main Program

Step 1: START

Step 2: Display the menu: 1. Full pyramid, 2. Inverted pyramid, 3. Diamond pattern, 4. Palindromic number pyramid, 5. Repeated-number pyramid.

Step 3: Read choice.

Step 4: Using a switch statement on choice:

 If choice is 1, call fullPyramid().

 If choice is 2, call invertedPyramid().

 If choice is 3, call diamondPattern().

 If choice is 4, call palindromicPyramid().

 If choice is 5, call repeatedNumberPyramid().

Step 5: STOP

Function fullPyramid()

Step 1: Read n .

Step 2: Repeat for $i = 1$ to n :

 Repeat for $j = 1$ to $n - i$: print two blank spaces.

 Repeat for $j = 1$ to $2i - 1$: print an asterisk.

 Move to a new line.

Step 3: Return to the calling function.

Function invertedPyramid()

Step 1: Read n .

Step 2: Repeat for $i = 1$ to n :

Repeat for $j = 1$ to $i - 1$: print two blank spaces.

Repeat for $j = 1$ to $2(n - i) + 1$: print an asterisk.

Move to a new line.

Step 3: Return to the calling function.

Function `diamondPattern()`

Step 1: Read n , the number of rows of the upper half.

Step 2: Repeat for $i = 1$ to n :

Print $n - i$ blank places, then $2i - 1$ asterisks, then move to a new line.

Step 3: Repeat for $i = n - 1$ down to 1 :

Print $n - i$ blank places, then $2i - 1$ asterisks, then move to a new line.

Step 4: Return to the calling function.

Function `palindromicPyramid()`

Step 1: Read n .

Step 2: Repeat for $i = 1$ to n :

Repeat for $j = 1$ to $n - i$: print two blank spaces.

Repeat for $j = 1$ to i : print the value of j .

Repeat for $j = i - 1$ down to 1 : print the value of j .

Move to a new line.

Step 3: Return to the calling function.

Function `repeatedNumberPyramid()`

Step 1: Read n .

Step 2: Repeat for $i = 1$ to n :

Repeat for $j = 1$ to $n - i$: print two blank spaces.

Repeat for $j = 1$ to $2i - 1$: print the value of i .

Move to a new line.

Step 3: Return to the calling function.

3. Test Case Table

#	Menu Choice	Input	Expected Output	Actual Output	Result
1	1. Full pyramid	$n = 4$	Rows of 1, 3, 5 and 7 asterisks, centred	Same	PASS
2	2. Inverted pyramid	$n = 4$	Rows of 7, 5, 3 and 1 asterisks, centred	Same	PASS
3	3. Diamond pattern	$n = 4$	7 rows in all, the middle row having 7 asterisks	7 rows, middle row 7	PASS
4	4. Palindromic pyramid	$n = 4$	1 / 1 2 1 / 1 2 3 2 1 / 1 2 3 4 3 2 1	Same	PASS
5	5. Repeated-number pyramid	$n = 4$	1 / 2 2 2 / 3 3 3 3 3 / 4 4 4 4 4 4	Same	PASS
6	1. Full pyramid	$n = 1$	A single asterisk, the smallest pyramid	Same	PASS

#	Menu Choice	Input	Expected Output	Actual Output	Result
7	3. Diamond pattern	n = 2	3 rows in all, the middle row having 3 asterisks	3 rows, middle row 3	PASS

4. Source Code

pyramid.c

```
/* Menu driven program that uses a switch statement to print the
   pyramid or diamond pattern chosen by the user. Each pattern is
   written in its own function that takes nothing and returns nothing,
   and every pattern is made with nested loops. No arrays are used. */

#include <stdio.h>

void fullPyramid(void);
void invertedPyramid(void);
void diamondPattern(void);
void palindromicPyramid(void);
void repeatedNumberPyramid(void);

int main(void)
{
    int choice;

    printf("1. Full pyramid\n");
    printf("2. Inverted pyramid\n");
    printf("3. Diamond pattern\n");
    printf("4. Palindromic number pyramid\n");
    printf("5. Repeated-number pyramid\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1: fullPyramid();          break;
        case 2: invertedPyramid();      break;
        case 3: diamondPattern();       break;
        case 4: palindromicPyramid();   break;
        case 5: repeatedNumberPyramid(); break;
    }

    return 0;
}

/* Row i has 2i - 1 asterisks with n - i blank places in front. Each
   blank is two spaces wide, the same width that "*" takes, so the
   pyramid comes out properly centred. */
void fullPyramid(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i; j++)
            printf(" ");
        for (j = 1; j <= 2 * i - 1; j++)
            printf("* ");
        printf("\n");
    }
}

/* The same pyramid stood on its head. The widest row comes first, so
   the blanks grow while the asterisks shrink. */
void invertedPyramid(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i - 1; j++)
            printf(" ");
        for (j = 1; j <= 2 * i - 1; j++)
            printf("* ");
        printf("\n");
    }
}
```

```

        for (j = 1; j <= 2 * (n - i) + 1; j++)
            printf("* ");
        printf("\n");
    }
}

/* A full pyramid of n rows for the upper half, and an inverted one of
   n - 1 rows below it. The widest row is not repeated, so the whole
   diamond has 2n - 1 rows and its middle row has 2n - 1 asterisks. */
void diamondPattern(void)
{
    int n, i, j;

    printf("Enter the number of rows for the upper half: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i; j++)
            printf(" ");
        for (j = 1; j <= 2 * i - 1; j++)
            printf("* ");
        printf("\n");
    }
    for (i = n - 1; i >= 1; i--) {
        for (j = 1; j <= n - i; j++)
            printf(" ");
        for (j = 1; j <= 2 * i - 1; j++)
            printf("* ");
        printf("\n");
    }
}

/* Row i counts up from 1 to i and then back down to 1, which gives
   2i - 1 numbers in the row. */
void palindromicPyramid(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i; j++)
            printf(" ");
        for (j = 1; j <= i; j++)
            printf("%d ", j);
        for (j = i - 1; j >= 1; j--)
            printf("%d ", j);
        printf("\n");
    }
}

/* Row i prints the row number itself 2i - 1 times, so the shape is the
   same as the full pyramid but filled with numbers instead of stars. */
void repeatedNumberPyramid(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

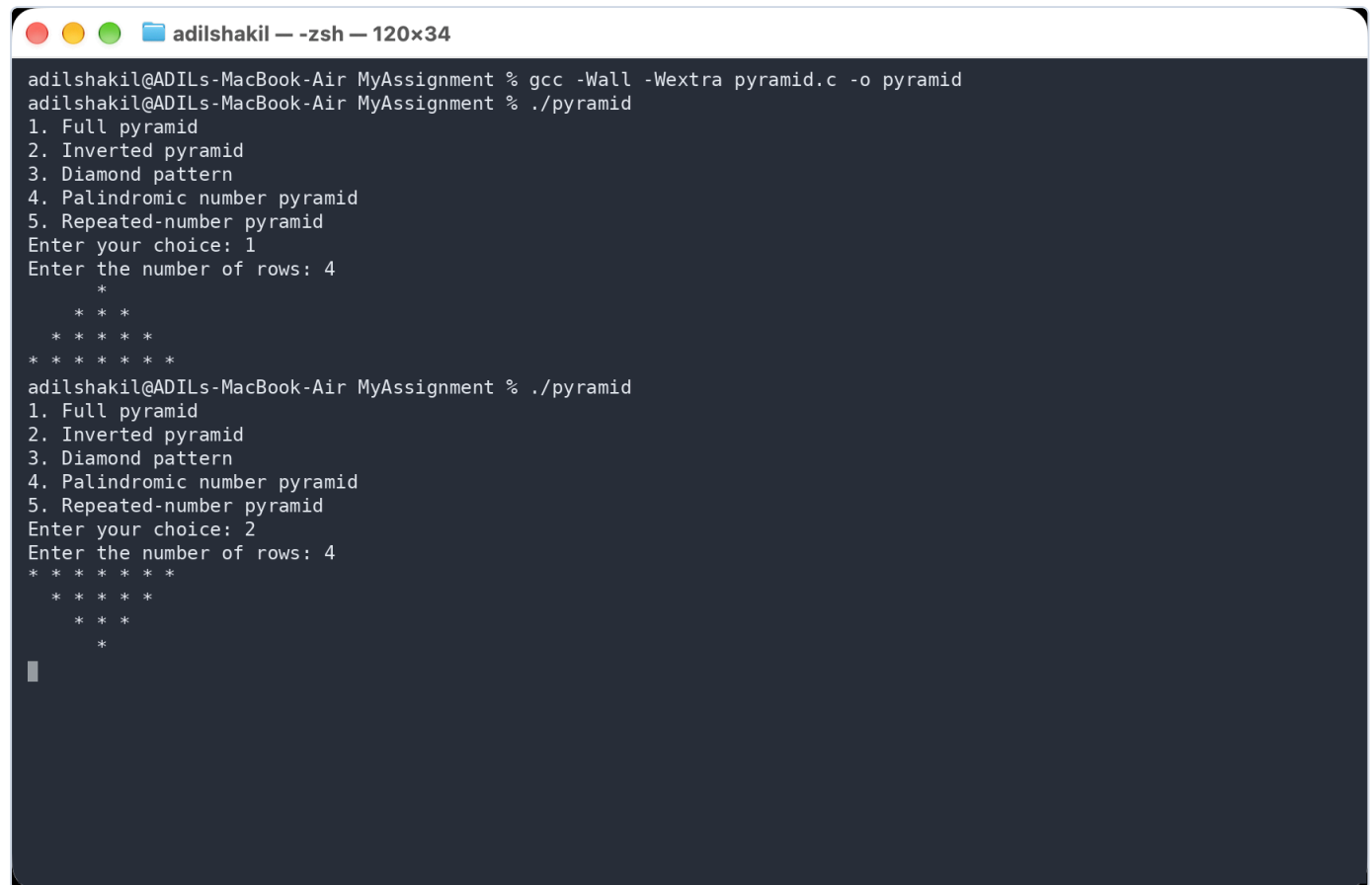
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i; j++)
            printf(" ");
        for (j = 1; j <= 2 * i - 1; j++)
            printf("%d ", i);
        printf("\n");
    }
}

```

5. Compilation and Output

I compiled it with `gcc -Wall -Wextra pyramid.c -o pyramid` so that even the small warnings would show up, and it compiled clean in the first attempt itself. The screenshots below are of the runs I did on my own machine.

Full Pyramid and Inverted Pyramid



```
adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra pyramid.c -o pyramid
adilshakil@ADILs-MacBook-Air MyAssignment % ./pyramid
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
Enter your choice: 1
Enter the number of rows: 4
*
* * *
* * * * *
* * * * * *
adilshakil@ADILs-MacBook-Air MyAssignment % ./pyramid
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
Enter your choice: 2
Enter the number of rows: 4
* * * * * *
* * * * *
* * *
*
```

Diamond Pattern and Palindromic Number Pyramid

```
adilshakil@ADILs-MacBook-Air MyAssignment % ./pyramid
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
Enter your choice: 3
Enter the number of rows for the upper half: 4
  *
 * * *
* * * * *
* * * * *
 * * *
  *

adilshakil@ADILs-MacBook-Air MyAssignment % ./pyramid
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
Enter your choice: 4
Enter the number of rows: 4
  1
 1 2 1
1 2 3 2 1
1 2 3 4 3 2 1
█
```

Repeated-Number Pyramid, and the Smallest Inputs

```
adilshakil@ADILs-MacBook-Air MyAssignment % ./pyramid
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
Enter your choice: 5
Enter the number of rows: 4
  1
 2 2 2
 3 3 3 3
4 4 4 4 4 4
adilshakil@ADILs-MacBook-Air MyAssignment % ./pyramid
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
Enter your choice: 1
Enter the number of rows: 1
*

adilshakil@ADILs-MacBook-Air MyAssignment % ./pyramid
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
Enter your choice: 3
Enter the number of rows for the upper half: 2
 *
* * *
 *
```

All seven test cases came out right and the five patterns match the sample outputs given in the question exactly. The thing that took me the longest was the diamond. My first version repeated the widest row twice, because I ran the lower half from n down to 1 instead of from $n - 1$, and then the diamond had $2n$ rows instead of $2n - 1$. Starting the second loop at $n - 1$ fixed it, and for $n = 4$ the diamond now comes out in exactly 7 rows with 7 asterisks in the middle row, which is what the question asks for.